```
#                                          "2" => {"title" => "Second
Chapter"}}}}

params.require(:book).permit(:title, chapters_attributes: [:title])
```

4.5.4 Outside the Scope of Strong Parameters

The strong parameter API was designed with the most common use cases in mind. It is not meant as a silver bullet to handle all your whitelisting problems. However you can easily mix the API with your own code to adapt to your situation.

Imagine a scenario where you have parameters representing a product name and a hash of arbitrary data associated with that product, and you want to whitelist the product name attribute but also the whole data hash. The strong parameters API doesn't let you directly whitelist the whole of a nested hash with any keys, but you can use the keys of your nested hash to declare what to whitelist:

```
def product_params
  params.require(:product).permit(:name, data:
params[:product][:data].try(:keys))
end
```

5 Session

Your application has a session for each user in which you can store small amounts of data that will be persisted between requests. The session is only available in the controller and the view and can use one of a number of different storage mechanisms:

- `ActionDispatch::Session::CookieStore` - Stores everything on the client.
- `ActionDispatch::Session::CacheStore` - Stores the data in the Rails cache.
- `ActionDispatch::Session::ActiveRecordStore` - Stores the data in a database using Active Record. (require `activerecord-session_store` gem).
- `ActionDispatch::Session::MemCacheStore` - Stores the data in a memcached cluster (this is a legacy implementation; consider using CacheStore instead).

All session stores use a cookie to store a unique ID for each session (you must use a cookie, Rails will not allow you to pass the session ID in the URL as this is less secure).

For most stores, this ID is used to look up the session data on the server, e.g. in a database table. There is one exception, and that is the default and recommended session store - the CookieStore - which stores all session data in the cookie itself (the ID is still available to you if you need it). This has the advantage of being very lightweight and it requires zero setup in a new application in order to use the session. The cookie data is cryptographically signed to make it tamper-proof. And it is also encrypted so anyone with access to it can't read its contents. (Rails will not accept it if it has been edited).

The CookieStore can store around 4kB of data - much less than the others - but this is usually enough. Storing large amounts of data in the session is discouraged no matter which session store your application uses. You should especially avoid storing complex objects (anything other than basic Ruby objects, the most common example being model instances) in the session, as the server might not be able to reassemble them between requests, which will result in an error.

If your user sessions don't store critical data or don't need to be around for long periods (for instance if you just use the flash for messaging), you can consider using `ActionDispatch::Session::CacheStore`. This